

HDOC 20

Name: KESHAV MITTAL **SAP ID:** 590032934 **Batch:** 16

1. Test Case Table

TC ID	Scenario Description	Inputs (global_id, m, n, code, credit, grade)	Expected Output
TC-01	Valid baseline inputs	global_id: 1001, m: 1, n: 2 C1: code:101, credit:4, grade:O C2: code:102, credit:3, grade:A	SGPA: 9.57 CGPA: 9.57 Total Credits: 7
TC-02	Multiple semesters	global_id: 1002, m: 2 Sem 1: n:1 (code:11, cred:2, grd:B) Sem 2: n:1 (code:12, cred:4, grd:C)	Sem 1 SGPA: 8.00 Sem 2 SGPA: 7.00 CGPA: 7.33 (Total Credits: 6)
TC-03	Invalid global_id (boundary)	global_id: -5 (Error), then 1003 m: 1, n: 1, code:10, credit:2, grade:P	Error displayed, asks again. SGPA: 4.00, CGPA: 4.00
TC-04	Invalid grade string input	global_id: 1004, m: 1, n: 1 code: 103, credit: 3, grade: X (Error), then A	Error displayed, asks again. SGPA: 9.00, CGPA: 9.00
TC-05	Invalid counts (m and n <= 0)	global_id: 1005, m: 0 (Error), then 1 n: -2 (Error), then 1 code: 104, credit: 4, grade: D	Errors displayed, asks again. SGPA: 6.00, CGPA: 6.00

2. C Program Source Code

```
#include <stdio.h>

int main() {
    int global_id, m, n, semester, course, code, credit, GP;
    int semester_TC, semester_TGC, cumulative_TC = 0, cumulative_TGC = 0;
    char grade;
    float SGPA, CGPA;

    // Step 3-4: Get and validate global_id
    do {
        printf("Enter global_id (> 0): ");
        scanf("%d", &global_id);
        if (global_id <= 0) {
            printf("Error: global_id must be positive.\n");
        }
    } while (global_id <= 0);

    // Step 5-6: Get and validate m
    do {
        printf("Enter number of semesters m (> 0): ");
        scanf("%d", &m);
        if (m <= 0) {
            printf("Error: number of semesters must be positive.\n");
        }
    } while (m <= 0);

    // Step 7: cumulative variables initialized in declaration

    // Step 8: Loop for m semesters
    for (semester = 1; semester <= m; semester++) {
        semester_TC = 0;
        semester_TGC = 0;

        printf("\n--- Entering data for Semester %d ---\n", semester);

        // Step 8.2-8.3: Get and validate n
        do {
            printf("Enter number of courses n (> 0): ");
            scanf("%d", &n);
            if (n <= 0) {
                printf("Error: number of courses must be positive.\n");
            }
        } while (n <= 0);

        // Step 8.4: Loop for n courses
        for (course = 1; course <= n; course++) {
            printf("\nCourse %d:\n", course);
```

```

// Validate code
do {
    printf("Enter course code (> 0): ");
    scanf("%d", &code);
    if (code <= 0) printf("Error: Invalid code.\n");
} while (code <= 0);

// Validate credit
do {
    printf("Enter course credit (> 0): ");
    scanf("%d", &credit);
    if (credit <= 0) printf("Error: Invalid credit.\n");
} while (credit <= 0);

// Validate grade
int valid_grade = 0;
do {
    printf("Enter grade (O, A, B, C, D, E, P): ");
    scanf(" %c", &grade); // Space before %c handles newline

    if (grade == 'O' || grade == 'A' || grade == 'B' || grade == 'C' ||
        grade == 'D' || grade == 'E' || grade == 'P') {
        valid_grade = 1;
    } else {
        printf("Error: Invalid grade. Try again.\n");
    }
} while (!valid_grade);

// Step 8.4.7: Find GP
switch (grade) {
    case 'O': GP = 10; break;
    case 'A': GP = 9; break;
    case 'B': GP = 8; break;
    case 'C': GP = 7; break;
    case 'D': GP = 6; break;
    case 'E': GP = 5; break;
    case 'P': GP = 4; break;
}

// Step 8.4.8 & 8.4.9
semester_TGC += (GP * credit);
semester_TC += credit;
}

// Step 8.5 & 8.6: Calculate and display SGPA
SGPA = (float)semester_TGC / semester_TC;
printf("\n>> Semester %d SGPA: %.2f <<\n", semester, SGPA);

// Step 8.7 & 8.8: Add to cumulative totals
cumulative_TGC += semester_TGC;
cumulative_TC += semester_TC;
}

```

```
// Step 9 & 10: Calculate CGPA and display final stats
CGPA = (float)cumulative_TGC / cumulative_TC;

printf("\n=====\\n");
printf("FINAL REPORT\\n");
printf("Global ID: %d\\n", global_id);
printf("Total Semesters (m): %d\\n", m);
printf("Cumulative Credits (TC): %d\\n", cumulative_TC);
printf("CGPA: %.2f\\n", CGPA);
printf("=====\\n");

return 0; // Step 11: Stop

}
```